

AERODYNE

Helium-Filled Flying Dolphin Balloon

Wireless Remote Control System with NRF24L01

Grade: 12 | Level: Advanced

Overview

About the Project

Aerodyne is a remotely controlled helium-filled dolphin-shaped balloon capable of navigating through indoor spaces. The balloon is driven by three DC motors — two for horizontal navigation (left and right) and one for vertical movement (ascend and descend). The system uses two Arduino Nano microcontrollers communicating wirelessly via the nRF24L01 radio module. A Python-based control console, called the AeroDyn Console, runs on a computer and sends commands serially to the transmitter unit, which then relays them wirelessly to the receiver mounted on the balloon.

Problem Statement

Conventional remote-controlled aerial vehicles require complex and expensive radio hardware or Wi-Fi infrastructure. There is a growing interest in lightweight, low-cost, and creative robotics platforms, especially for educational and demonstration purposes. A helium balloon-based system removes the need for heavy lift mechanisms, making the engineering challenge primarily one of navigation, communication, and control. Aerodyne addresses this by creating a fun, functional, and educational flying platform that teaches key concepts in wireless communication, motor control, and embedded systems.

Proposed Solution

The proposed solution uses a helium-filled dolphin balloon as the lift platform. Three DC motors handle directional movement. An Arduino Nano on the transmitter side receives commands from a Python program on the host computer via USB serial and forwards them wirelessly using an nRF24L01 module. The receiver, also an Arduino Nano with an nRF24L01, decodes the incoming commands and drives the motors via two L293D motor driver ICs. A 7805 voltage regulator provides stable 5V power to the onboard electronics from an external battery. It also supports autopilot

Supported movement commands:

- CMD_FORWARD — Both lateral motors spin forward
- CMD_LEFT — Left motor reverses, right motor moves forward (turns left)
- CMD_RIGHT — Right motor reverses, left motor moves forward (turns right)
- CMD_ASCEND — Vertical motor lifts the balloon
- CMD_DESCEND — Vertical motor lowers the balloon
- CMD_STOP — All motors stop

Prerequisites

Before starting this project, students should be familiar with the following concepts:

- Basic Arduino programming (setup(), loop(), pinMode, digitalWrite, analogWrite)
- Serial communication concepts (UART, baud rate, Serial.begin, Serial.readStringUntil)
- SPI communication protocol used by the nRF24L01 module
- H-bridge motor driver operation (L293D)
- PWM (Pulse Width Modulation) for speed control
- Python serial programming with the pyserial library
- Basic understanding of RF wireless communication
- Voltage regulation using the 7805 IC

Components Required

Component	Quantity	Unit (approx. INR)
Arduino Nano	2	500 each
nRF24L01 Wireless Module	2	200 each
L293D Motor Driver IC	2	80 each
DC Motor (small)	3	150 each
7805 Voltage Regulator	1	20
Helium Balloon (dolphin)	1	600–1000
USB Cable (Nano compatible)	2	100 each
9V Battery / Power Bank	1	200–500
Connecting Wires (pack)	1	100
Breadboard / PCB	2	150 each

Arduino Nano

The Arduino Nano is a compact microcontroller board based on the ATmega328P chip. It operates at 5V and 16 MHz, providing 14 digital I/O pins (6 PWM capable) and 8 analog inputs. In this project, two Arduino Nano boards are used — one for the transmitter and one for the receiver. The Nano communicates with the nRF24L01 module via SPI and with the host computer via USB serial.



nRF24L01 Wireless Module

The nRF24L01 is a 2.4 GHz single-chip radio transceiver designed for ultra-low power wireless communication. It uses the SPI protocol to interface with the Arduino and supports data rates up to 2 Mbps. The module uses a fixed address ("00001" in this project) and can operate in both transmitting and receiving modes under software control. One module is set to transmit (stopListening) and the other to receive (startListening).



L293D Motor Driver IC

The L293D is a dual H-bridge motor driver IC capable of driving two DC motors independently. Two L293D ICs are used in this project — one handles the left and right motors, and the other manages the middle (vertical) motor. Each IC accepts logic-level control signals from the Arduino and drives the motors at higher current levels. Speed is controlled using the PWM signal on the enable pin (connected to Arduino D3).



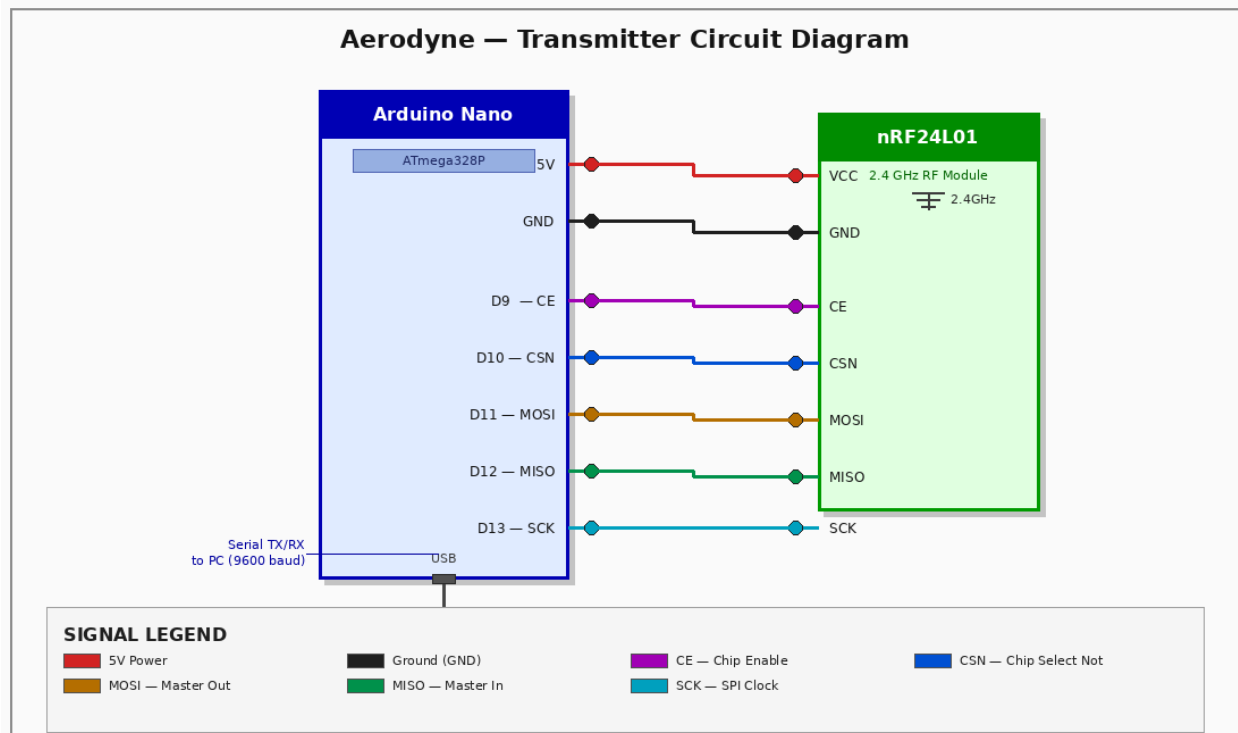
7805 Voltage Regulator

The 7805 is a linear voltage regulator that provides a stable +5V output from an input of 7V to 35V. In the receiver unit, a 9V battery or power bank is stepped down to 5V to power the Arduino Nano and the nRF24L01 module. The 7805 is important for ensuring stable operation of the electronics even as the battery voltage fluctuates.

Circuit Diagram

Transmitter Circuit

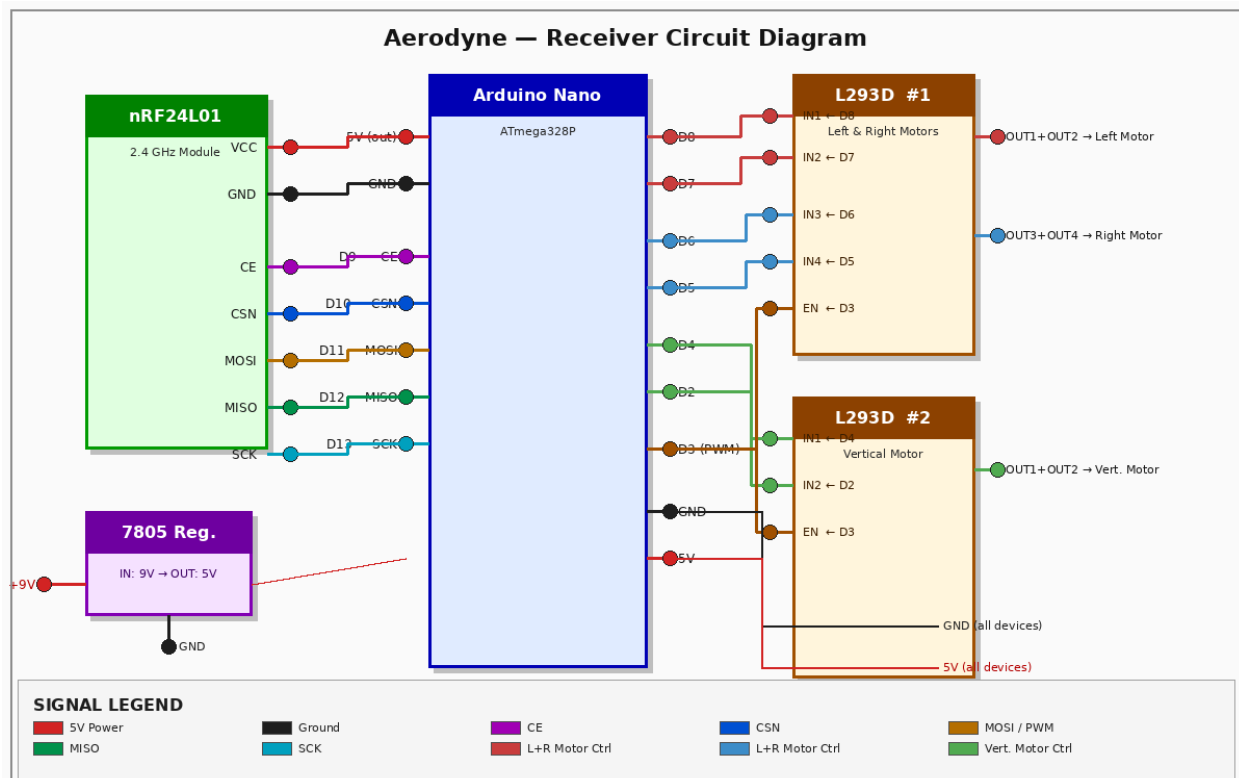
The transmitter consists of an Arduino Nano connected to an nRF24L01 module via SPI. The Arduino receives commands from the host computer via USB serial, converts them to radio packets, and transmits them wirelessly.



Arduino Nano Pin	nRF24L01 Pin	Function
D9	CE	Chip Enable
D10	CSN	Chip Select Not
D11	MOSI	SPI Master Out
D12	MISO	SPI Master In
D13	SCK	SPI Clock
3.3V	VCC	Power Supply
GND	GND	Ground
USB / D0/D1	—	Serial Comm to PC

Receiver Circuit

The receiver unit is mounted on the balloon and consists of an Arduino Nano, an nRF24L01 module, two L293D motor driver ICs for the three motors, and a 7805 voltage regulator for stable power supply from a 9V battery.

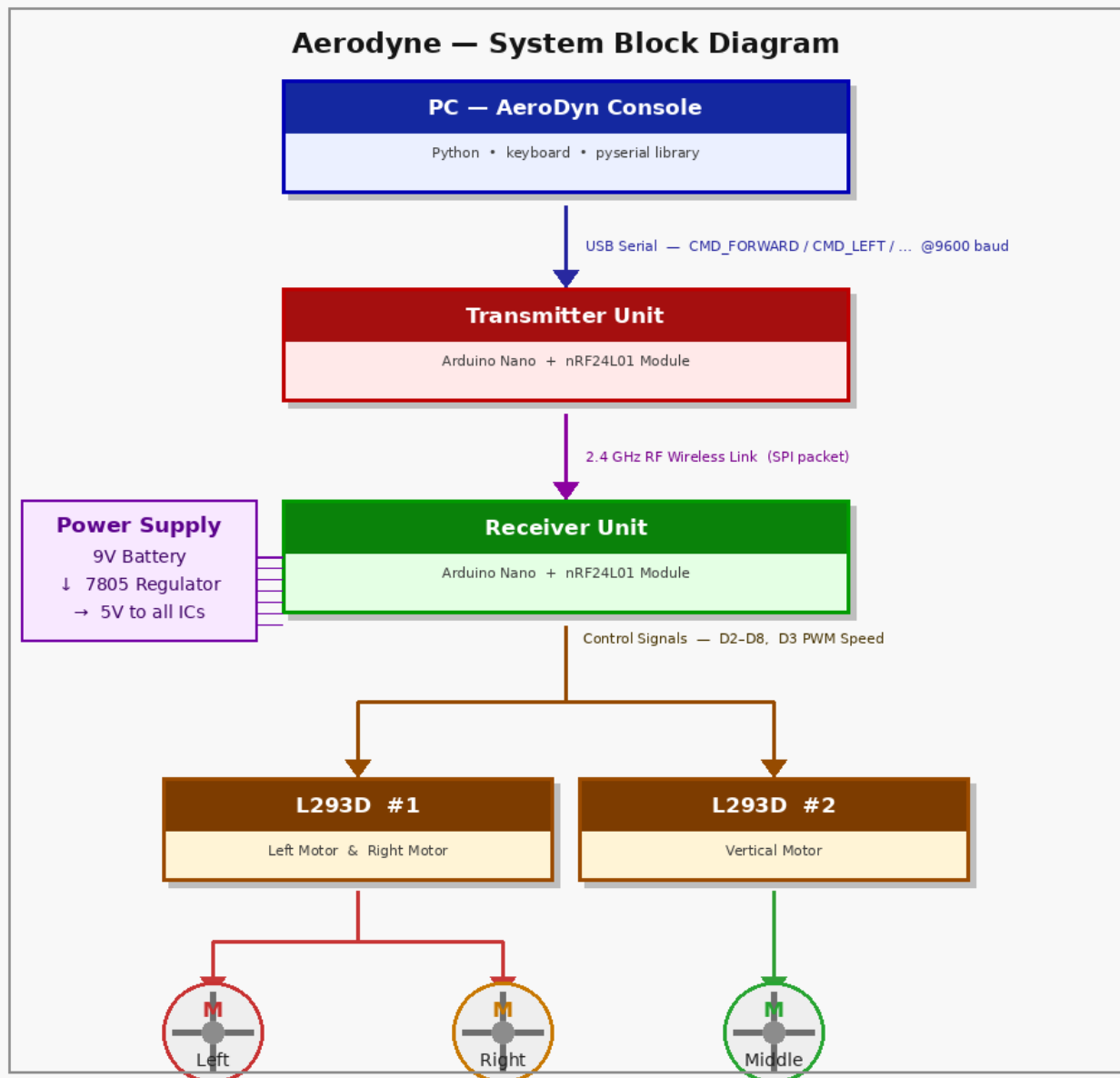


Arduino Nano Pin	Connected To	Function
D9	nRF24L01 CE	SPI Chip Enable
D10	nRF24L01 CSN	SPI Chip Select
D11	nRF24L01 MOSI	SPI Data Out
D12	nRF24L01 MISO	SPI Data In
D13	nRF24L01 SCK	SPI Clock
D8, D7	L293D #1 IN1, IN2	Left Motor Control
D6, D5	L293D #1 IN3, IN4	Right Motor Control

D4, D2	L293D #2 IN1, IN2	Middle Motor Control
D3 (PWM)	L293D EN Pins	Speed Control
5V (from 7805)	All VCC Pins	Regulated Power
GND	All GND Pins	Common Ground

Block Diagram

The Aerodyne system works across three layers: the software controller, the transmitter hardware, and the receiver hardware on the balloon.



Source Code

Transmitter Firmware (transmitter.ino)

The transmitter firmware runs on the Arduino Nano connected to the host PC. It listens for serial input from the Python console, reads the command string, and transmits it via the nRF24L01 module.

```
#include <SPI.h>

#include <nRF24L01.h>

#include <RF24.h>


RF24 radio(9, 10);                // CE=D9, CSN=D10

const byte address[6] = "00001"; // Fixed channel address

char msg_aerodyn_tx[32] = "";


void setup() {

    Serial.begin(9600);

    radio.begin();

    radio.openWritingPipe(address);

    radio.setPALevel(RF24_PA_LOW);

    radio.stopListening();        // Set to transmit mode
}


void loop() {

    if (Serial.available()) {

        String input = Serial.readStringUntil('\n');

        input.trim();

        input.toCharArray(msg_aerodyn_tx, 32);
```

```

        radio.write(&msg_aerodyn_tx, sizeof(msg_aerodyn_tx));
    }
}

```

Receiver Firmware (receptor2.ino)

The receiver firmware listens for incoming radio packets and translates them into motor commands. Each recognized command string triggers the appropriate motor function.

```

#include <SPI.h>

#include <nRF24L01.h>

#include <RF24.h>

#include <string.h>

RF24 radio(9, 10);

const byte address[6] = "00001";

int motorPinLeft1 = 8, motorPinLeft2 = 7;

int motorPinRight1 = 6, motorPinRight2 = 5;

int motorPinMiddle1 = 4, motorPinMiddle2 = 2;

int speedControlPin = 3;

int speed = 50;

char msg_aerodyn_rx[32];

void setup() {

    Serial.begin(9600);

    radio.begin();

```

```

    radio.openReadingPipe(1, address);

    radio.setPALevel(RF24_PA_LOW);

    radio.startListening();           // Set to receive mode

    for (int i = 2; i <= 8; i++) {

        pinMode(i, OUTPUT);

        digitalWrite(i, LOW);

    }
}

void forward() {

    digitalWrite(motorPinLeft1, HIGH);  digitalWrite(motorPinLeft2, LOW);

    digitalWrite(motorPinRight1, HIGH); digitalWrite(motorPinRight2, LOW);

    analogWrite(speedControlPin, speed);

}

void left() {

    digitalWrite(motorPinLeft1, LOW);   digitalWrite(motorPinLeft2, HIGH);

    digitalWrite(motorPinRight1, HIGH); digitalWrite(motorPinRight2, LOW);

    analogWrite(speedControlPin, speed);

}

void right() {

    digitalWrite(motorPinLeft1, HIGH);  digitalWrite(motorPinLeft2, LOW);

    digitalWrite(motorPinRight1, LOW);  digitalWrite(motorPinRight2, HIGH);

    analogWrite(speedControlPin, speed);

}

```

```

void ascend() { digitalWrite(motorPinMiddle1, HIGH); digitalWrite(motorPinMiddle2,
LOW); analogWrite(speedControlPin, speed); }

void descend() { digitalWrite(motorPinMiddle1, LOW); digitalWrite(motorPinMiddle2,
HIGH); analogWrite(speedControlPin, speed); }

void stop()    { for (int i = 2; i <= 8; i++) digitalWrite(i, LOW); }

void loop() {

    if (radio.available()) {

        memset(msg_aerodyn_rx, 0, sizeof(msg_aerodyn_rx));

        radio.read(&msg_aerodyn_rx, sizeof(msg_aerodyn_rx));

        msg_aerodyn_rx[31] = '\0';

        if      (strcmp(msg_aerodyn_rx, "CMD_FORWARD") == 0) forward();

        else if (strcmp(msg_aerodyn_rx, "CMD_LEFT")    == 0) left();

        else if (strcmp(msg_aerodyn_rx, "CMD_RIGHT")   == 0) right();

        else if (strcmp(msg_aerodyn_rx, "CMD_ASCEND")  == 0) ascend();

        else if (strcmp(msg_aerodyn_rx, "CMD_DESCEND") == 0) descend();

        else if (strcmp(msg_aerodyn_rx, "CMD_STOP")    == 0) stop();

    }

}

```

AeroDyn Console — Python Controller (aerodyn_console.py)

The Python console runs on the host PC and sends commands via the USB serial port to the transmitter Arduino. It supports both manual keyboard control and an autopilot mode.

```
import keyboard, serial, config, time, random

ser = serial.Serial(config.SERIAL_PORT, config.BAUD_RATE)

time.sleep(2)

while True:

    if keyboard.is_pressed("up"):

        ser.write(config.MSG_FORWARD)      # Arrow Up → Forward

    elif keyboard.is_pressed("left"):

        ser.write(config.MSG_LEFT)         # Arrow Left → Turn Left

    elif keyboard.is_pressed("right"):

        ser.write(config.MSG_RIGHT)        # Arrow Right → Turn Right

    elif keyboard.is_pressed("down"):

        for _ in range(4):

            ser.write(config.MSG_DESCEND)   # Arrow Down → Descend (x4)

            time.sleep(2)

    elif keyboard.is_pressed("space"):

        ser.write(config.MSG_STOP)          # Space → Stop all motors

    elif keyboard.is_pressed("tab"):

        # TAB → Autopilot Mode

        while True:

            cmd = random.choice([config.MSG_FORWARD,

                                config.MSG_LEFT,

                                config.MSG_RIGHT,

                                config.MSG_DESCEND])

            ser.write(cmd)
```

```
time.sleep(2)

ser.write(config.MSG_STOP)

time.sleep(2)

if keyboard.is_pressed("esc"):

    break
```

Code Explanation

Transmitter — How It Works

The transmitter sketch initializes the radio in writing (TX) mode using `radio.stopListening()`. The main loop checks if data is available on the Serial port. When a newline-terminated command arrives from the Python console, it is read using `Serial.readStringUntil('\n')`, trimmed of whitespace, copied into a 32-byte character array, and transmitted with `radio.write()`. The nRF24L01 handles all the low-level RF encoding, preamble, CRC, and retransmission automatically.

Receiver — Motor Functions

The receiver starts in listening mode (`radio.startListening()`). In the loop, when `radio.available()` returns true, the 32-byte message buffer is cleared with `memset()` to prevent stale data issues, then read with `radio.read()`. The null terminator is explicitly set at position 31 for safety. The message is compared to known command strings using `strcmp()`, and the corresponding motor function is called. Each motor function uses `digitalWrite()` pairs to set the H-bridge direction and `analogWrite()` on the enable pin for speed.

AeroDyn Console — Key Controls

Key	Command Sent	Action
↑ Up Arrow	CMD_FORWARD	Balloon moves forward
← Left Arrow	CMD_LEFT	Balloon turns left
→ Right Arrow	CMD_RIGHT	Balloon turns right
↓ Down Arrow	CMD_DESCEND × 4	Balloon descends gradually
Space	CMD_STOP	All motors stop
Tab	Autopilot (random)	Auto random navigation
Esc	—	Exit autopilot mode

config.py — Configuration File

The Python project relies on a config.py file that must define the serial port, baud rate, and command constants. A typical config.py looks like this:

```
# config.py

SERIAL_PORT = "COM3"          # Change to your Arduino's port

BAUD_RATE = 9600

MSG_FORWARD = b"CMD_FORWARD\n"

MSG_LEFT    = b"CMD_LEFT\n"

MSG_RIGHT   = b"CMD_RIGHT\n"

MSG_DESCEND = b"CMD_DESCEND\n"

MSG_STOP    = b"CMD_STOP\n"

MSG_LOAD    = "AeroDyn Console Ready."
```

Implementation and Output

Procedure

1. Inflate the dolphin balloon with helium and attach the motor assembly frame underneath it.
2. Mount the receiver Arduino Nano, two L293D ICs, the nRF24L01 module, and the 7805 regulator on a lightweight PCB or mini breadboard. Attach this to the balloon frame.
3. Connect the 9V battery to the 7805 regulator. Verify the output is 5V before connecting electronics.
4. Connect all motors to the L293D outputs as per the pin table. Verify motor direction by testing each motor individually.
5. Upload the receptor2.ino firmware to the receiver Arduino Nano.
6. Assemble the transmitter — connect the nRF24L01 to the Arduino Nano as per the transmitter circuit.
7. Upload the transmitter.ino firmware to the transmitter Arduino Nano.
8. Connect the transmitter Arduino to the PC via USB.
9. Install Python dependencies: `pip install pyserial keyboard`
10. Create config.py and set the correct COM port for your transmitter Arduino.
11. Run the console: `python aerodyn_console.py`
12. Use arrow keys to fly the balloon. Press Tab to activate autopilot mode, Esc to exit autopilot.

Expected Output

Once the system is operational, pressing an arrow key on the keyboard causes the balloon to move in the corresponding direction within 100–200ms, limited mainly by serial latency and RF transmission delay. The autopilot mode alternates random commands every 2 seconds with stops in between, causing the balloon to drift around the room autonomously.

Troubleshooting

Problem	Possible Solution
No serial connection	Check COM port in config.py. Try a different USB cable. Ensure the correct port appears in Device Manager / Arduino IDE.
NRF not communicating	Verify SPI wiring (CE=D9, CSN=D10). Ensure both modules use the same address. Check that the nRF module is powered at 3.3V (or use a 3.3V regulator as some modules are sensitive to 5V).
Motor spins wrong direction	Swap the two motor wires (OUT1 and OUT2) connected to that motor, or swap IN1/IN2 logic in the code.
Balloon not responding	Check that both Arduinos have the correct firmware uploaded. Verify the battery is charged. Check all GND connections are common.
Python console crash	Ensure the keyboard and pyserial libraries are installed. Run with administrator / sudo privileges if keyboard input is not detected.
Motors spin but weak	Increase the speed variable in receptor2.ino (max 255). Ensure the battery can supply enough current for all three motors simultaneously.

Cost Estimation

Component	Price (INR)
Arduino Nano × 2	₹1,000
nRF24L01 Module × 2	₹400
L293D Motor Driver IC × 2	₹160
DC Motors × 3	₹450
7805 Voltage Regulator	₹20
Helium Balloon (dolphin)	₹20000
9V Battery + Wires + PCB	₹350
Miscellaneous (tape, frame, zip ties)	₹100
Total Estimated Cost	₹22,480

*** Helium cylinder charges extra**

Conclusion

The Aerodyne project demonstrates a practical and engaging application of wireless communication, embedded systems, and motor control. By building a flying dolphin balloon that can be steered from a laptop keyboard, students gain hands-on experience with SPI protocol, H-bridge motor drivers, RF modules, serial communication, and Python hardware interfacing — all key skills for advanced electronics and robotics projects. The modular architecture of Aerodyne — with a clear separation between the Python controller, transmitter firmware, and receiver firmware — also teaches good software design principles. The autopilot feature adds an element of autonomous behaviour, hinting at more complex applications such as path planning and sensor-guided navigation that can be explored as extensions.

Future improvements could include adding an ultrasonic sensor for obstacle avoidance, an IMU for orientation feedback, a mobile app controller, or even a camera module for FPV (first-person view) flight. Aerodyne is not just a fun demonstration — it is a launchpad for deeper exploration into the field of aerial robotics.